3

The Euclidean Algorithm

Integers and polynomials with coefficients in a field behave similarly in many respects. Often—but not always—the algorithms for both types of objects are quite similar, and sometimes one can find a common abstraction of both domains, and it is then sufficient to design one algorithm for this generalization to solve both problems in one fell swoop. In this chapter, the Euclidean domain covers the structural similarities between gcd computations for integers and polynomials. Typically, in such a situation the polynomial version is slightly simpler, and in Chapter 6, we will meet polynomial subresultants which have no integer analog at all.

3.1. Euclidean domains

The Euclidean Algorithm for the two integers 126 and 35 works as follows:

$$\begin{aligned} 126 &= 3\cdot 35 + 21, \\ 35 &= 1\cdot 21 + 14, \\ 21 &= 1\cdot 14 + 7, \\ 14 &= 2\cdot 7, \end{aligned} \tag{1}$$

and 7 is the greatest common divisor of 126 and 35. One of the most important applications is for exact arithmetic on rational numbers, where one has to simplify $35/126$ to $5/18$ in order to keep the numbers small.

This algorithm can also be adapted to work for polynomials. It is convenient to use the following general scenario, which captures both situations under one umbrella. The reader may always think of R as being either the integers or polynomials. The algebraic terminology is explained in Chapter 25.

DEFINITION 3.1. *An integral domain R with a function $d: R \longrightarrow \mathbb{N} \cup \{-\infty\}$ is a* ***Euclidean domain*** *if for all $a, b \in R$ with $b \neq 0$, we can divide a by b with remainder, so that*

$$\text{there exist } q, r \in R \text{ such that } a = qb + r \text{ and } d(r) < d(b). \tag{2}$$

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

We say that $q = a$ quo b is the ***quotient*** *and $r = a$ rem b the* ***remainder****, although q and r need not be unique. Such a d is called a* ***Euclidean function*** *on R.*

EXAMPLE 3.2. (i) $R = \mathbb{Z}$ and $d(a) = |a| \in \mathbb{N}$. Here the quotient and the remainder can be made unique by the additional requirement that $r \geq 0$.

(ii) $R = F[x]$, where F is a field, and $d(a) = \deg a$. We define the degree of the zero polynomial to be $-\infty$. It is easy to show uniqueness of the quotient and the remainder in this case (Section 2.4).

(iii) $R = \mathbb{Z}[i] = \{a + ib : a, b \in \mathbb{Z}\}$, the ring of Gaussian integers, with $i = \sqrt{-1}$, and $d(a + ib) = a^2 + b^2$ (Exercise 3.19).

(iv) R a field, and $d(a) = 1$ if $a \neq 0$ and $d(0) = 0$. $\diamond$

The value $d(b)$ is never $-\infty$ except possibly when $b = 0$.

DEFINITION 3.3. *Let R be a ring and $a, b, c \in R$. Then c is a* ***greatest common divisor*** *(or* gcd*) of a and b if*

(i) $c \mid a$ and $c \mid b$,

(ii) if $d \mid a$ and $d \mid b$, then $d \mid c$, for all $d \in R$.

Similarly, c is called a ***least common multiple*** *(or* lcm*) of a and b if*

(i) $a \mid c$ and $b \mid c$,

(ii) if $a \mid d$ and $b \mid d$, then $c \mid d$, for all $d \in R$.

A ***unit*** *$u \in R$ is any element with a multiplicative inverse $v \in R$, so that $uv = 1$. The elements a and b are* ***associate*** *if $a = ub$ for a unit $u \in R$; we then write $a \sim b$.*

For example, 3 is a gcd of 12 and 15, and 60 is an lcm of 12 and 15 in $\mathbb{Z}$. In general, neither the gcd nor the lcm are unique, but all gcds of a and b are precisely the associates of one of them, and similarly for the lcms. The only units in $\mathbb{Z}$ are 1 and -1, and 3 and -3 are all gcds of 12 and 15 in $\mathbb{Z}$. For $R = \mathbb{Z}$, we may define $\gcd(a, b)$ as the unique nonnegative greatest common divisor and $\operatorname{lcm}(a, b)$ as the unique nonnegative least common multiple of a and b. As an example, for negative $a \in \mathbb{Z}$ we then have $\gcd(a, a) = \gcd(a, 0) = -a$. We say that two integers a, b are **coprime** (or relatively prime) if their gcd is a unit.

Greatest common divisors and least common multiples need not exist in an arbitrary ring; for an example, see Section 25.2. In the following section, however, we will prove that a gcd always exists in a Euclidean domain, and as a consequence an lcm also exists.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

LEMMA 3.4. *The* gcd *in* $\mathbb{Z}$ *has the following properties, for all* $a, b, c \in \mathbb{Z}$.

(i) $\gcd(a,b) = |a| \iff a \mid b$,

(ii) $\gcd(a,a) = \gcd(a,0) = |a|$ *and* $\gcd(a,1) = 1$,

(iii) $\gcd(a,b) = \gcd(b,a)$ *(commutativity)*,

(iv) $\gcd(a,\gcd(b,c)) = \gcd(\gcd(a,b),c)$ *(associativity)*,

(v) $\gcd(c\cdot a, c\cdot b) = |c|\cdot\gcd(a,b)$ *(distributivity)*,

(vi) $|a| = |b| \Longrightarrow \gcd(a,c) = \gcd(b,c)$.

For a proof, see Exercise 3.3. Because of the associativity, we may write

$$\gcd(a_1,\ldots,a_n) = \gcd(a_1,\gcd(a_2,\ldots,\gcd(a_{n-1},a_n)\ldots)).$$

The following algorithm computes greatest common divisors in an arbitrary Euclidean domain. The nonuniqueness of quotient and remainder complicates the description a bit. For now, "$\gcd(a,b)$" stands for any element which is a gcd of a and b, and we assume some functions quo and rem which associate to any two elements $a, b \in R$ with $b \neq 0$ unique $q = a$ quo b and $r = a$ rem b in R with $a = qb + r$ and $d(r) < d(b)$. Section 3.4 fixes the notation so that gcd has only a single value.

ALGORITHM 3.5 Traditional Euclidean Algorithm.
Input: $f, g \in R$, where R is a Euclidean domain with Euclidean function d.
Output: A greatest common divisor of f and g.

1. $r_0 \longleftarrow f, \quad r_1 \longleftarrow g$
2. $i \longleftarrow 1$
   while $r_i \neq 0$ **do** $r_{i+1} \longleftarrow r_{i-1}$ rem $r_i, \quad i \longleftarrow i+1$
3. **return** r_{i-1}.

For $f = 126$ and $g = 35$, the algorithm works precisely as illustrated at the beginning of this section.

3.2. The Extended Euclidean Algorithm

The following extension of Algorithm 3.5 computes not only the gcd but also a representation of it as a linear combination of the inputs. It generalizes the representation

$$7 = 21 - 1\cdot 14 = 21 - (35 - 1\cdot 21) = 2\cdot(126 - 3\cdot 35) - 35 = 2\cdot 126 - 7\cdot 35,$$

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

which is obtained by reading the lines of (1) from the bottom up. This important method is called the **Extended Euclidean Algorithm** and works in any Euclidean domain. In various incarnations, it plays a central role throughout this book.

ALGORITHM 3.6 Traditional Extended Euclidean Algorithm.
Input: $f, g \in R$, where R is a Euclidean domain.
Output: $\ell \in \mathbb{N}$, $r_i, s_i, t_i \in R$ for $0 \le i \le \ell+1$, and $q_i \in R$ for $1 \le i \le \ell$, as computed below.

1. $r_0 \longleftarrow f, \quad s_0 \longleftarrow 1, \quad t_0 \longleftarrow 0,$
   $r_1 \longleftarrow g, \quad s_1 \longleftarrow 0, \quad t_1 \longleftarrow 1$
2. $i \longleftarrow 1$
   while $r_i \neq 0$ **do**
   $q_i \longleftarrow r_{i-1}$ quo r_i
   $r_{i+1} \longleftarrow r_{i-1} - q_i r_i$
   $s_{i+1} \longleftarrow s_{i-1} - q_i s_i$
   $t_{i+1} \longleftarrow t_{i-1} - q_i t_i$
   $i \longleftarrow i+1$
3. $\ell \longleftarrow i-1$
   return ℓ, r_i, s_i, t_i for $0 \le i \le \ell+1$, and q_i for $1 \le i \le \ell$

We note that the algorithm terminates because the $d(r_i)$ are strictly decreasing nonnegative integers for $1 \le i \le \ell$, where d is the Euclidean function on R. The elements r_i for $0 \le i \le \ell+1$ are the **remainders** and the q_i for $1 \le i \le \ell$ are the **quotients** in the traditional (Extended) Euclidean Algorithm. The elements r_i, s_i, and t_i form the ***i*th row** in the traditional Extended Euclidean Algorithm, for $0 \le i \le \ell+1$. The central property is that $s_i f + t_i g = r_i$ for all i; in particular, $s_\ell f + t_\ell g = r_\ell$ is a gcd of f and g (see Lemma 3.8 below). We will see later that *all* other intermediate results computed by the algorithm are useful for various tasks in computer algebra.

EXAMPLE 3.7. (i) As in (1), we consider $R = \mathbb{Z}$, $f = 126$, and $g = 35$. The following table illustrates the computation.

i	q_i	r_i	s_i	t_i
0		126	1	0
1	3	35	0	1
2	1	21	1	−3
3	1	14	−1	4
4	2	7	2	−7
5		0	−5	18

We can read off row 4 that $\gcd(126, 35) = 7 = 2 \cdot 126 + (-7) \cdot 35$.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

(ii) $R = \mathbb{Q}[x]$, $f = 18x^3 - 42x^2 + 30x - 6$, $g = -12x^2 + 10x - 2$. Then the computation of the traditional Extended Euclidean Algorithm goes as follows. Row $i+1$ is obtained from the two preceding ones by first computing the quotient $q_i = r_{i-1}$ quo r_i and then for each of the three remaining columns by subtracting the quotient times the entry in row i of that column from the entry in row $i-1$.

i	q_i	r_i	s_i	t_i
0		$18x^3 - 42x^2 + 30x - 6$	1	0
1	$-\frac{3}{2}x + \frac{9}{4}$	$-12x^2 + 10x - 2$	0	1
2	$-\frac{8}{3}x + \frac{4}{3}$	$\frac{9}{2}x - \frac{3}{2}$	1	$\frac{3}{2}x - \frac{9}{4}$
3		0	$\frac{8}{3}x - \frac{4}{3}$	$4x^2 - 8x + 4$

We have $\ell = 2$, and from row 2, we find that a gcd of f and g is

$$\frac{9}{2}x - \frac{3}{2} = 1 \cdot (18x^3 - 42x^2 + 30x - 6) + \left(\frac{3}{2}x - \frac{9}{4}\right)(-12x^2 + 10x - 2). \diamond$$

For a global view of the algorithm, it is convenient to consider the matrices

$$R_0 = \begin{pmatrix} s_0 & t_0 \\ s_1 & t_1 \end{pmatrix}, \quad Q_i = \begin{pmatrix} 0 & 1 \\ 1 & -q_i \end{pmatrix} \text{ for } 1 \le i \le \ell$$

in $R^{2\times 2}$, and $R_i = Q_i \cdots Q_1 R_0$ for $0 \le i \le \ell$. The following lemma collects some invariants of the traditional Extended Euclidean Algorithm.

LEMMA 3.8. *For $0 \le i \le \ell$, we have*

(i) $R_i \cdot \begin{pmatrix} f \\ g \end{pmatrix} = \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix}$,

(ii) $R_i = \begin{pmatrix} s_i & t_i \\ s_{i+1} & t_{i+1} \end{pmatrix}$,

(iii) $\gcd(f,g) \sim \gcd(r_i, r_{i+1}) \sim r_\ell$,

(iv) $s_i f + t_i g = r_i$ *(this also holds for $i = \ell + 1$)*,

(v) $s_i t_{i+1} - t_i s_{i+1} = (-1)^i$,

(vi) $\gcd(r_i, t_i) \sim \gcd(f, t_i)$,

(vii) $f = (-1)^i (t_{i+1} r_i - t_i r_{i+1})$, $g = (-1)^{i+1}(s_{i+1} r_i - s_i r_{i+1})$,

with the convention that $r_{\ell+1} = 0$.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

PROOF. For (i) and (ii) we proceed by induction on i. The case $i = 0$ is clear from step 1 of the algorithm, and we may assume $i \geq 1$. Then

$$Q_i \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & -q_i \end{pmatrix} \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = \begin{pmatrix} r_i \\ r_{i-1} - q_i r_i \end{pmatrix} = \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix},$$

and (i) follows from $R_i = Q_i R_{i-1}$ and the induction hypothesis. Similarly, (ii) follows from

$$Q_i \begin{pmatrix} s_{i-1} & t_{i-1} \\ s_i & t_i \end{pmatrix} = \begin{pmatrix} s_i & t_i \\ s_{i+1} & t_{i+1} \end{pmatrix}$$

and the induction hypothesis.

For (iii), let $i \in \{0, \ldots, \ell\}$. We conclude from (i) that

$$\begin{pmatrix} r_\ell \\ 0 \end{pmatrix} = Q_\ell \cdots Q_{i+1} R_i \begin{pmatrix} f \\ g \end{pmatrix} = Q_\ell \cdots Q_{i+1} \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix}.$$

Comparing the first entry on both sides, we see that r_ℓ is a linear combination of r_i and r_{i+1}, and hence any common divisor of r_i and r_{i+1} divides r_ℓ. On the other hand, $\det Q_i = -1$ and the matrix Q_i is invertible over R, with inverse

$$Q_i^{-1} = \begin{pmatrix} q_i & 1 \\ 1 & 0 \end{pmatrix},$$

and hence

$$\begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = Q_{i+1}^{-1} \cdots Q_\ell^{-1} \begin{pmatrix} r_\ell \\ 0 \end{pmatrix}.$$

Thus both r_i and r_{i+1} are divisible by r_ℓ, and $r_\ell \sim \gcd(r_i, r_{i+1})$. In particular, this is true for $i = 0$, so that $\gcd(f, g) \sim \gcd(r_0, r_1) \sim r_\ell$.

The claim (iv) follows immediately from (i) and (ii), and (v) follows from (ii) by taking determinants:

$$\begin{aligned} s_i t_{i+1} - t_i s_{i+1} &= \det \begin{pmatrix} s_i & t_i \\ s_{i+1} & t_{i+1} \end{pmatrix} = \det R_i \\ &= \det Q_i \cdots \det Q_1 \cdot \det \begin{pmatrix} s_0 & t_0 \\ s_1 & t_1 \end{pmatrix} = (-1)^i. \end{aligned}$$

In particular, this implies that $\gcd(s_i, t_i) \sim 1$ and that R_i is invertible. Now let $p \in R$ be a divisor of t_i. If $p \mid f$, then clearly $p \mid s_i f + t_i g = r_i$. On the other hand, if $p \mid r_i$, then p also divides $s_i f = r_i - t_i g$, and hence p divides f since s_i and t_i are coprime. This proves (vi). For (vii), we multiply both sides of (i) by R_i^{-1} and obtain

$$\begin{pmatrix} r_0 \\ r_1 \end{pmatrix} = R_i^{-1} \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = (-1)^i \begin{pmatrix} t_{i+1} & -t_i \\ -s_{i+1} & s_i \end{pmatrix} \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix},$$

using (ii) and (v), and the claim follows by writing this out as a system of linear equations. □

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

COROLLARY 3.9.
Any two elements f, g of a Euclidean domain R have a gcd $h \in R$, *and it is expressible as a linear combination $h = sf + tg$ with $s, t \in R$.*

3.3. Cost analysis for $\mathbb{Z}$ and $F[x]$

We want to analyze the cost of the traditional Extended Euclidean Algorithm 3.6 for $f, g \in R$ with $n = d(f) \geq d(g) = m \geq 0$. The number ℓ of division steps is obviously bounded by $\ell \leq d(g) + 1$. We investigate the two important cases $R = F[x]$ and $R = \mathbb{Z}$ separately, starting with $R = F[x]$, where F is a field, and $d(a) = \deg a$ as usual.

We let $n_i = \deg r_i$ for $0 \leq i \leq \ell + 1$, with $r_{\ell+1} = 0$. Then $n_0 = n \geq n_1 = m > n_2 > \cdots > n_\ell$, and $\deg q_i = n_{i-1} - n_i$ for $1 \leq i \leq \ell$. According to Section 2.4, we can divide the polynomial r_{i-1} of degree n_{i-1} by the polynomial r_i of degree $n_i \leq n_{i-1}$ with remainder using at most $(2n_i + 1)(n_{i-1} - n_i + 1)$ additions and multiplications plus one inversion in F. (Recall that we count a subtraction as an addition.) Thus the total cost for the traditional Euclidean Algorithm, that is, for computing only the r_i and q_i, including a gcd of f and g, is

$$\sum_{1 \leq i \leq \ell} (2n_i + 1)(n_{i-1} - n_i + 1) \tag{3}$$

additions and multiplications plus $\ell \leq m + 1$ inversions in F. We first evaluate the expression above for the **normal** case where the degree drops exactly by 1 at each step, so that $n_i = m - i + 1$ for $2 \leq i \leq \ell = m + 1$, and later show that this is the worst case. Since $n_{i-1} - n_i + 1 = 2$ for $i \geq 2$ and $n_1 = m$, (3) simplifies to

$$\begin{aligned} &(2m+1)(n-m+1) + 2 \sum_{2 \leq i \leq m+1} (2(m-i+1)+1) \\ &\quad = \quad (2m+1)(n-m+1) + 2(m^2 - m) + 2m = 2nm + n + m + 1. \end{aligned} \tag{4}$$

We now consider the sum $\sigma(n_0, n_1, \ldots, n_\ell)$ in (3) as a function of the integers $n_0 \geq n_1 > n_2 > \cdots > n_\ell \geq 0$ and show that it increases if we insert an additional integer $n_{j-1} > k > n_j$ for some $j \in \{2, \ldots, \ell\}$ or append some integer $n_\ell > k \geq 0$:

$$\begin{aligned} &\sigma(n_0, \ldots, n_{j-1}, k, n_j, \ldots, n_\ell) - \sigma(n_0, \ldots, n_\ell) \\ &\quad = (2k+1)(n_{j-1} - k + 1) + (2n_j + 1)(k - n_j + 1) - (2n_j + 1)(n_{j-1} - n_j + 1) \\ &\quad = 2(n_{j-1} - k)(k - n_j) + 2k + 1 > 0. \end{aligned}$$

A similar argument works for $n_\ell > k \geq 0$. Proceeding inductively, we find that $\sigma(n_0, n_1, n_2, \ldots, n_\ell) < \sigma(n_0, n_1, n_1 - 1, \ldots, 1, 0)$, and conclude that the bound (4) is valid in any case.

It remains to determine the cost for computing s_i, t_i on the way.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

LEMMA 3.10.

$$\deg s_i = \sum_{2 \le j < i} \deg q_j = n_1 - n_{i-1} \text{ for } 2 \le i \le \ell + 1, \tag{5}$$

$$\deg t_i = \sum_{1 \le j < i} \deg q_j = n_0 - n_{i-1} \text{ for } 1 \le i \le \ell + 1. \tag{6}$$

PROOF. We only prove the first equality; the second can be verified in the same way (Exercise 3.21 (i)). We show (5) and

$$\deg s_{i-1} < \deg s_i \text{ for } 2 \le i \le \ell + 1 \tag{7}$$

by simultaneous induction on i. For $i = 2$, we find that $s_2 = s_0 - q_1 s_1 = 1 - q_1 \cdot 0 = 1$, and $\deg s_1 = -\infty < 0 = \deg s_2$. Now we assume that $i \ge 2$ and that the claims are already proven for $2 \le j \le i$. Then, by the induction hypothesis (7), we have

$$\deg s_{i-1} < \deg s_i < n_{i-1} - n_i + \deg s_i = \deg(q_i s_i),$$

which implies that

$$\deg s_{i+1} = \deg(s_{i-1} - q_i s_i) = \deg q_i + \deg s_i > \deg s_i,$$

and

$$\deg s_{i+1} = \deg q_i + \deg s_i = \sum_{2 \le j < i} \deg q_j + \deg q_i = \sum_{2 \le j < i+1} \deg q_j,$$

where we used the induction hypothesis (5). □

THEOREM 3.11.

The traditional Extended Euclidean Algorithm 3.6 for polynomials $f, g \in F[x]$ with $\deg f = n \ge \deg g = m$ can be performed with

- *at most $m + 1$ inversions and $2nm + O(n)$ additions and multiplications in F if only the quotients q_i and the remainders r_i are needed,*
- *at most $m + 1$ inversions and $6nm + O(n)$ additions and multiplications in F for computing all results.*

PROOF. The first claim has already been shown, and it remains to analyze the additional cost for computing the s_i and t_i. At each step, the computation of $t_{i+1} = t_{i-1} - q_i t_i$ requires at most $2 \deg q_i \deg t_i + \deg q_i + \deg t_i + 1$ field operations for the product (Section 2.3), plus at most $\deg t_{i+1} + 1$ operations for the subtraction. Using Lemma 3.10, we obtain

$$\sum_{2 \le i \le \ell} \Big(2(n_{i-1} - n_i)(n_0 - n_{i-1}) + 2(n_0 - n_i + 1) \Big)$$

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

additions and multiplications in F, plus $n-m+1$ for $i=1$. In the normal case, this becomes

$$\begin{aligned} n-m+1+\sum_{2\le i\le m+1}\Big(2(n-(m-i+2))+2(n-(m-i+1)+1)\Big) \\ = n-m+1+4\sum_{2\le i\le m+1}(n-m+i-1) \\ = n-m+1+4m(n-m)+2(m^2+m)\in 4nm-2m^2+O(n). \end{aligned}$$

A similar argument as above shows that the normal case is the worst case, so that the bound is valid in general. Finally, Exercise 3.22 (i) shows that the cost for the s_i's is at most $2(m^2+m)$, and the claim follows. □

In Chapter 11, we will find a much faster algorithm for the gcd.

Now we sketch the cost analysis when $R=\mathbb{Z}$ and $d(a)=|a|$. We may assume that $f=r_0\ge g=r_1>r_2\cdots>r_\ell\ge 0$, so that $q_i\ge 1$ for all i, and represent all numbers in 2^{64}-ary standard representation (Section 2.1). Then the length $\lambda(a)$ of a positive integer a is $\lambda(a)=\lfloor(\log a)/64\rfloor+1$, where log is the binary logarithm. But now the bound corresponding to what we used for polynomials, namely $\ell\le d(g)+1=g+1=(2^{64})^{(\log g)/64}+1\le 2^{64\lambda(g)}$, on the number of division steps in the Euclidean Algorithm for the pair $(f,g)\in\mathbb{N}^2$ is exponential in the input size $\lambda(f)+\lambda(g)$ (if $\lambda(f)$ is not much bigger than $\lambda(g)$) and hence rather useless. We can in fact prove a polynomial upper bound on ℓ, as follows. For $1\le i\le\ell$, we have

$$r_{i-1}=q_ir_i+r_{i+1}\ge r_i+r_{i+1}>2r_{i+1}.$$

Thus

$$\prod_{2\le i<\ell}r_{i-1}>2^{\ell-2}\prod_{2\le i<\ell}r_{i+1}$$

if $\ell>2$, and $r_{\ell-1}\ge 2$ implies that

$$2^{\ell-2}<\frac{r_1r_2}{r_{\ell-1}r_\ell}<\frac{r_1^2}{2},$$

$$\ell\le\lfloor 2\log r_1\rfloor+1=\left\lfloor 128\frac{\log g}{64}\right\rfloor+1\le 128\left(\left\lfloor\frac{\log g}{64}\right\rfloor+1\right)=128\lambda(g).$$

This bound can still be improved. For $N\in\mathbb{N}$ and $f,g\in\mathbb{Z}$ with $N\ge f>g>0$, the largest possible number of division steps ℓ for (f,g) is the one where all the quotients are equal to 1, so that f and g are the two largest successive Fibonacci numbers up to N. As an example, the Euclidean Algorithm for $(f,g)=(13,8)$ computes

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

$$\begin{aligned} 13 &= 1 \cdot 8 + 5, \\ 8 &= 1 \cdot 5 + 3, \\ 5 &= 1 \cdot 3 + 2, \\ 3 &= 1 \cdot 2 + 1, \\ 2 &= 2 \cdot 1. \end{aligned}$$

The nth **Fibonacci number** F_n (with $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$) is approximately $\phi^n/\sqrt{5}$, where $\phi = (1+\sqrt{5})/2 \approx 1.618$ is the **golden ratio** (Exercise 3.28). Thus the following holds for the number ℓ of division steps for $(f,g) = (F_{n+1}, F_n)$ if $n \geq 1$:

$$\ell = n - 1 \approx \log_\phi \sqrt{5} g - 1 \in 1.441 \log g + O(1). \tag{8}$$

The average number of division steps for (f,g) when g is fixed and f varies is

$$\ell \approx \frac{12(\ln 2)^2}{\pi^2} \log g \approx 0.584 \log g.$$

Now that we have a good upper bound for the number of steps in the Euclidean Algorithm, we look at the cost for each step. First we consider the cost for one division step. Let $a > b > 0$ be integers and $a = qb + r$ with $q, r \in \mathbb{N}$ and $0 \leq r < b$. According to Section 2.4, computing q and r takes $O((\lambda(a) - \lambda(b)) \cdot \lambda(b))$ word operations, where $\lambda(a)$ and $\lambda(b)$ are the lengths of a and b in the standard representation, respectively.

Then setting $n = \lambda(f)$ and $m = \lambda(g)$, we obtain—by analogy with (4)—that the total cost for performing the traditional Euclidean Algorithm (without computing the s_i and t_i) is $O(nm)$ word operations.

The following integer analog of Lemma 3.10 is proven in Exercise 3.23.

LEMMA 3.12. *$|s_i| \leq \dfrac{g}{r_{i-1}}$ and $|t_i| \leq \dfrac{f}{r_{i-1}}$ for $1 \leq i \leq \ell + 1$.*

Lemma 3.12 yields analogous bounds for the length of s_i and t_i as in the polynomial case, and we have the following theorem, whose proof is left as Exercise 3.24.

THEOREM 3.13.
The traditional Extended Euclidean Algorithm 3.6 for positive integers f, g with $\lambda(f) = n \geq \lambda(g) = m$ can be performed with $O(nm)$ word operations.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

N	c_N/N^2
10	0.63
100	0.6087
1000	0.608383
10000	0.60794971
100000	0.6079301507

TABLE 3.1: The probabilities that two random positive integers below N are coprime.

We conclude this section with the following question: what is the probability that two random integers are coprime? More precisely, when N gets large and $c_N = \#\{1 \le x,y \le N : \gcd(x,y) = 1\}$, we are interested in the numerical value of c_N/N^2. Table 3.1 gives c_N/N^2 for some values of N; it seems to approach a limit which is a little larger than $3/5$. In fact, the value is

$$\frac{c_N}{N^2} \in \frac{6}{\pi^2} + O\Big(\frac{\log N}{N}\Big) \approx 0.6079271016 + O\Big(\frac{\log N}{N}\Big).$$

Interestingly, a similar approximation holds for the probability that a random integer is **squarefree**, so that it has no square divisor p^2:

$$\frac{\#\{1 \le x \le N : x \text{ is squarefree}\}}{N} \in \frac{6}{\pi^2} + O\Big(\frac{1}{\sqrt{N}}\Big).$$

Exercises 4.18 and 14.32 answer the corresponding questions for polynomials over a finite field.

In Figure 3.2, we see a two-dimensional coordinate system where the point $(x,y) \in \mathbb{N}^2$ for $x,y \le 200$ is colored white if $\gcd(x,y) = 1$ and gray otherwise. The intensity of a pixel is proportional to the number of prime factors in the gcd. The probability that two random integers below 200 are coprime is precisely the percentage of the area of the 200×200 pixels that is colored white. Thus about $3/5$ of all pixels are white, and about $2/5$ are gray.

If you hold the page horizontally in front of your eyes, you can see (almost) white horizontal and vertical lines corresponding to prime values of x and y, and dark lines through the origin corresponding to lines $ax = by$ with small integers a,b, the most clearly visible being the line $x = y$.

3.4. (Non-)Uniqueness of the gcd

The nonuniqueness of the gcd is a harmless nuisance from a mathematical point of view. But in software, we have to implement a *function* gcd with a unique output. In this section, we discuss one way of achieving this.

Since $\mathbb{Q}$ is a field, every nonzero rational number is a unit in $\mathbb{Q}$, and so $ua \sim a$ in $R = \mathbb{Q}[x]$ for all nonzero $u \in \mathbb{Q}$ and all $a \in R$. If we want to define a single

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

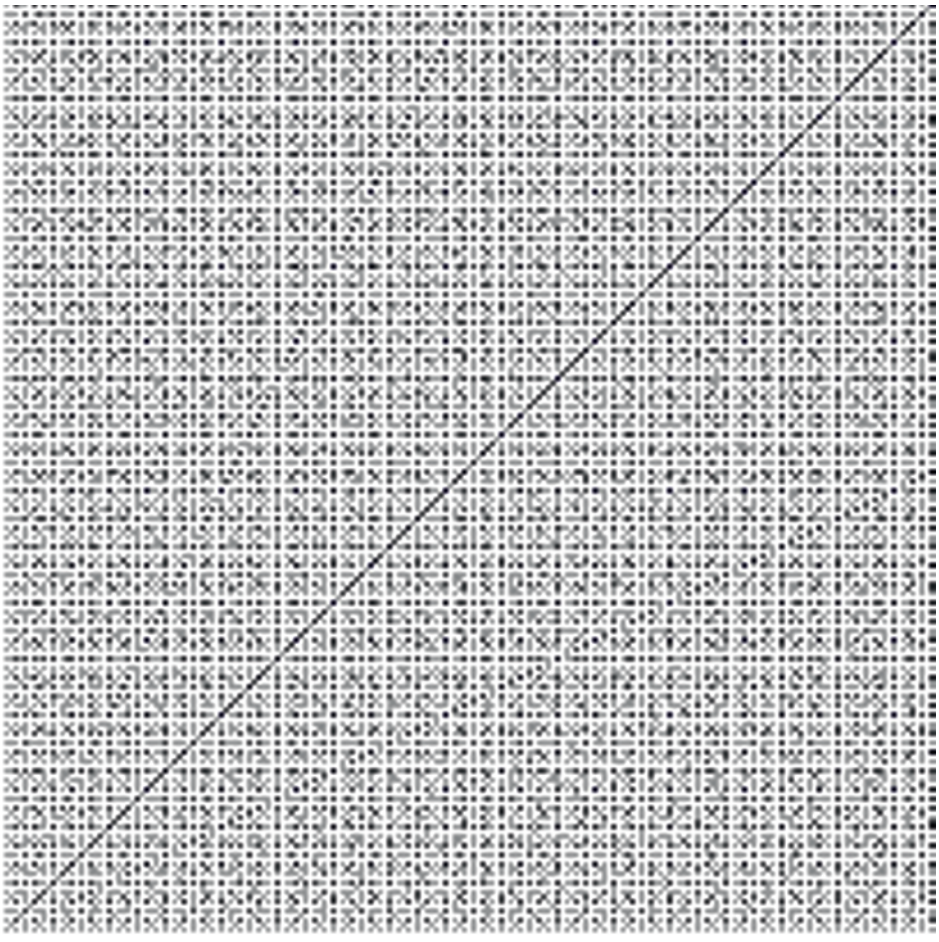


FIGURE 3.2: The greatest common divisors of x and y for $1 \leq x, y \leq 200$.

element $\gcd(f,g) \in \mathbb{Q}[x]$, which one should we choose? In other words, how do we choose *one* representative from among all the multiples of a? A reasonable choice is the **monic** polynomial, that is, the one with leading coefficient 1. Thus if $\mathrm{lc}(a) \in \mathbb{Q} \setminus \{0\}$ is the leading coefficient of $a \in \mathbb{Q}[x]$, then we take $\mathrm{normal}(a) = a/\mathrm{lc}(a)$ as the **normal form** of a. (This has nothing to do with the "normal EEA" on page 51.)

To make this work in an arbitrary Euclidean domain R, we assume that we have selected some normal form $\mathrm{normal}(a) \in R$ for every $a \in R$ so that $a \sim \mathrm{normal}(a)$. We call the unit $u \in R$ with $a = u \cdot \mathrm{normal}(a)$ the **leading unit** $\mathrm{lu}(a)$ of a. Moreover, we set $\mathrm{lu}(0) = 1$ and $\mathrm{normal}(0) = 0$. The following two properties are required:

- two elements of R have the same normal form if and only if they are associate,
- the normal form of a product is equal to the product of the normal forms.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

These properties in particular imply that the normal form of any unit is 1. We say that an element a in normal form, so that $\mathrm{lu}(a) = 1$, is **normalized**.

In our two main applications, integers and univariate polynomials over a field, we have natural normal forms. If $R = \mathbb{Z}$, $\mathrm{lu}(a) = \mathrm{sign}(a)$ if $a \neq 0$ and $\mathrm{normal}(a) = |a|$ defines a normal form, so that an integer is normalized if and only if it is nonnegative. When $R = F[x]$ for a field F, then letting $\mathrm{lu}(a) = \mathrm{lc}(a)$ (with the convention that $\mathrm{lu}(0) = 1$) and $\mathrm{normal}(a) = a/\mathrm{lc}(a)$ defines a normal form, and a nonzero polynomial is normalized if and only if it is monic.

Given such a normal form, we define $\gcd(a,b)$ to be the unique normalized associate of all greatest common divisors of a and b, and similarly $\mathrm{lcm}(a,b)$ as the normalized associate of all least common multiples of a and b. Thus $\gcd(a,b) > 0$ for $R = \mathbb{Z}$ and $\gcd(a,b)$ is monic for $R = F[x]$ if at least one of a,b is nonzero, and $\gcd(0,0) = 0$ in both cases. Lemma 3.4 then remains valid if we replace $|\cdot|$ by $\mathrm{normal}(\cdot)$.

In the polynomial case, it turns out that it is not only useful to have a normal form for the gcd, but to modify the traditional Euclidean Algorithm so that *all* the remainders r_i are normalized. In Chapter 6, we will see that for $R = \mathbb{Q}[x]$ the computations of the traditional Euclidean Algorithm produce remainders whose coefficients have huge numerators and denominators even for inputs of moderate size, and that the coefficients of the monic associates of the remainders are much smaller (see pages 143 and 185). In this book, we will often use the following variant of the traditional Extended Euclidean Algorithm 3.6 which works with these monic associates.

ALGORITHM 3.14 Extended Euclidean Algorithm (EEA).
Input: $f, g \in R$, where R is a Euclidean domain with a normal form.
Output: $\ell \in \mathbb{N}$, $\rho_i, r_i, s_i, t_i \in R$ for $0 \leq i \leq \ell+1$, and $q_i \in R$ for $1 \leq i \leq \ell$, as computed below.

1. $\rho_0 \longleftarrow \mathrm{lu}(f)$, $r_0 \longleftarrow \mathrm{normal}(f)$, $s_0 \longleftarrow \rho_0^{-1}$, $t_0 \longleftarrow 0$,
   $\rho_1 \longleftarrow \mathrm{lu}(g)$, $r_1 \longleftarrow \mathrm{normal}(g)$, $s_1 \longleftarrow 0$, $t_1 \longleftarrow \rho_1^{-1}$
2. $i \longleftarrow 1$
   while $r_i \neq 0$ **do**
   $q_i \longleftarrow r_{i-1}$ quo r_i
   $\rho_{i+1} \longleftarrow \mathrm{lu}(r_{i-1} - q_i r_i)$
   $r_{i+1} \longleftarrow (r_{i-1} - q_i r_i)/\rho_{i+1}$
   $s_{i+1} \longleftarrow (s_{i-1} - q_i s_i)/\rho_{i+1}$
   $t_{i+1} \longleftarrow (t_{i-1} - q_i t_i)/\rho_{i+1}$
   $i \longleftarrow i+1$
3. $\ell \longleftarrow i-1$
   return ℓ, ρ_i, r_i, s_i, t_i for $0 \leq i \leq \ell+1$, and q_i for $1 \leq i \leq \ell$

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

The elements r_i for $0 \le i \le \ell+1$ are the **remainders**, the q_i for $1 \le i \le \ell$ are the **quotients**, and the elements r_i, s_i, and t_i form the ***i*th row** in the Extended Euclidean Algorithm, for $0 \le i \le \ell+1$. The elements s_ℓ and t_ℓ, satisfying $s_\ell f + t_\ell g = \gcd(f,g)$, are the **Bézout coefficients** of f and g.

EXAMPLE 3.7 (continued). (ii) With monic remainders, the following quantities are computed.

i	q_i	ρ_i	r_i	s_i	t_i
0		18	$x^3 - \frac{7}{3}x^2 + \frac{5}{3}x - \frac{1}{3}$	$\frac{1}{18}$	0
1	$x - \frac{3}{2}$	-12	$x^2 - \frac{5}{6}x + \frac{1}{6}$	0	$-\frac{1}{12}$
2	$x - \frac{1}{2}$	$\frac{1}{4}$	$x - \frac{1}{3}$	$\frac{2}{9}$	$\frac{1}{3}x - \frac{1}{2}$
3		1	0	$-\frac{2}{9}x + \frac{1}{9}$	$-\frac{1}{3}x^2 + \frac{2}{3}x - \frac{1}{3}$

From row 2, we find that

$$\gcd(f,g) = x - \frac{1}{3} = \frac{2}{9}\cdot(18x^3 - 42x^2 + 30x - 6) + \left(\frac{1}{3}x - \frac{1}{2}\right)(-12x^2 + 10x - 2). \diamond$$

The matrices Q_i now become:

$$Q_i = \begin{pmatrix} 0 & 1 \\ \rho_{i+1}^{-1} & -q_i\rho_{i+1}^{-1} \end{pmatrix} \text{ for } 1 \le i \le \ell.$$

LEMMA 3.15. *(a) With the following modifications, all statements of Lemma 3.8 hold for the results of Algorithm 3.14.*

(iii) $\gcd(f,g) = \gcd(r_i, r_{i+1}) = r_\ell$,

(v) $s_i t_{i+1} - t_i s_{i+1} = (-1)^i(\rho_0 \cdots \rho_{i+1})^{-1}$,

(vi) $\gcd(r_i, t_i) = \gcd(f, t_i)$,

(vii) $f = (-1)^i \rho_0 \cdots \rho_{i+1}(t_{i+1}r_i - t_i r_{i+1})$, $g = (-1)^{i+1}\rho_0 \cdots \rho_{i+1}(s_{i+1}r_i - s_i r_{i+1})$.

(b) If $R = F[x]$ for a field F, $\deg f \ge \deg g$, and $n_i = \deg r_i$ for all i, then the degree formulas of Lemma 3.10 hold for the results of Algorithm 3.14 as well.

PROOF. With the following changes, the proof of Lemma 3.8 goes through:

$$Q_i \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ \rho_{i+1}^{-1} & -q_i\rho_{i+1}^{-1} \end{pmatrix} \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = \begin{pmatrix} r_i \\ (r_{i-1} - q_i r_i)\rho_{i+1}^{-1} \end{pmatrix} = \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix},$$

$$Q_i^{-1} = \begin{pmatrix} q_i & \rho_{i+1} \\ 1 & 0 \end{pmatrix},$$

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

$$\det Q_i \cdots \det Q_1 \cdot \det \begin{pmatrix} s_0 & t_0 \\ s_1 & t_1 \end{pmatrix} = (-1)^i (\rho_0 \cdots \rho_{i+1})^{-1},$$

$$\begin{pmatrix} r_0 \\ r_1 \end{pmatrix} = R_i^{-1} \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = (-1)^i (\rho_0 \cdots \rho_{i+1}) \begin{pmatrix} t_{i+1} & -t_i \\ -s_{i+1} & s_i \end{pmatrix} \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix}.$$

Statements (iii) and (vi) follow from the fact that all elements involved are normalized. The proof of (b) is left as Exercise 3.21 (ii). □

We conclude this section with a cost analysis of the EEA for polynomials. It turns out to be not more expensive than the traditional EEA.

THEOREM 3.16.
For the monic normal form $\operatorname{normal}(h) = h/\operatorname{lc}(h)$ *on* $F[x]$, *the Extended Euclidean Algorithm 3.14 for polynomials* $f, g \in F[x]$ *with* $\deg f = n \geq \deg g = m$ *can be performed with*

- *at most* $m+2$ *inversions and* $2nm + O(n)$ *additions and multiplications in* F *if only the quotients* q_i, *the remainders* r_i, *and the coefficients* ρ_i *are needed,*
- *at most* $m+2$ *inversions and* $6nm + O(n)$ *additions and multiplications in* F *for computing all results.*

PROOF. We proceed as in Section 3.3 and let $n_i = \deg r_i$ for $0 \leq i \leq \ell+1$, with $r_{\ell+1} = 0$, so that $\deg q_i = n_{i-1} - n_i$ for $1 \leq i \leq \ell$. Division with remainder of a monic polynomial a by a monic polynomial b is slightly cheaper than a general division; it takes only $2\deg b \cdot (\deg a - \deg b) + \deg b$ additions and multiplications in F. Thus the cost for computing the quotient q_i and the remainder on division of r_{i-1} by r_i is $2n_i(n_{i-1} - n_i) + n_i$. If $i < \ell$, we compute ρ_{i+1}^{-1} and multiply the remainder by it to obtain r_{i+1}, taking one inversion plus n_{i+1} multiplications. Thus the cost for computing all q_i and r_{i+1} for $1 \leq i \leq \ell$ is $\ell - 1 \leq m$ inversions and

$$\sum_{1 \leq i \leq \ell} \Big(2n_i(n_{i-1} - n_i) + n_i \Big) + \sum_{1 \leq i < \ell} n_{i+1}$$

additions and multiplications. For a normal degree sequence, where $n_i = m - i + 1$ for $1 \leq i \leq \ell = m+1$, this becomes

$$2m(n-m) + m + \sum_{2 \leq i \leq m+1} 4(m-i+1) = 2m(n-m) + m + 2(m^2 - m) = 2nm - m.$$

As in Section 3.3, the normal case is the worst case, normalizing f and g takes two inversions and $n+m$ multiplications, and (i) follows.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

As for division with remainder, multiplying a monic polynomial a by a polynomial b is a bit cheaper than a general multiplication; it takes only $2\deg a \cdot \deg b + \deg a$ additions and multiplications in F (Section 2.3). Thus computing $q_i t_i$ takes $2(n_{i-1}-n_i)(n-n_{i-1})+n_{i-1}-n_i$ operations, by Lemma 3.15 (b). Subtracting the product from t_{i-1} and multiplying the result by ρ_{i+1}^{-1} to obtain t_{i+1} takes another $2(n-n_i+1)$ additions and multiplications. Hence the cost for computing all t_i is

$$\sum_{1\le i\le \ell} \Big(2(n_{i-1}-n_i)(n-n_{i-1})+n_{i-1}-n_i+2(n-n_i+1)\Big)$$

additions and multiplications in F. In the normal case, this simplifies to

$$\begin{aligned}\sum_{1\le i\le m+1} &\Big(2(n-(m-i+2))+1+2(n-(m-i+1)+1)\Big)\\ &= m+1+\sum_{1\le i\le m+1} 4(n-m+i-1)\\ &= m+1+4(m+1)(n-m)+2(m^2+m) = 4nm-2m^2+4n-m+1.\end{aligned}$$

Exercise 3.22 (ii) shows that the cost for computing all s_i is at most $2m^2+3m+1$ in the normal case. Again, the normal case is the worst case, and (ii) follows. □

Theorem 6.53 (i) in Section 6.11 shows that in the polynomial case, the results of the traditional EEA and the results of Algorithm 3.14 are constant multiples of each other.

Taking the positive or monic gcd in $\mathbb{Z}$ or $F[x]$, respectively, is a reasonable solution to the nonuniqueness problem. However, when you implement computer algebra software, many other rings will be relevant, and often normalization is not compatible across domains. For example, $\gcd(-10x, 5x^2)$ is not really defined unless we specify the domain R in Definition 3.3. Using R as a subscript, we have—under normalization—$\gcd_{\mathbb{Q}[x]}(-10x, 5x^2) = x$, and $\pm 5x$ are candidates for $\gcd_{\mathbb{Z}[x]}(-10x, 5x^2)$. A computer algebra system has to make an assumption here, unless it allows the user to specify the domain; for our example, usually $\mathbb{Z}[x]$ is assumed.

If R is a domain with a normal form normal_R, then we get one for the polynomial ring $R[x]$ by setting

$$\mathrm{normal}_{R[x]}(f) = \frac{\mathrm{normal}_R(\mathrm{lc}(f))}{\mathrm{lc}(f)} \cdot f,$$

where $\mathrm{lc}(f)$ is the leading coefficient of f (Exercise 3.8 (iii)). Inductively, this defines a normal form, and hence a unique gcd, for multivariate polynomials over $\mathbb{Z}$ or over any field.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

Notes. 3.1. The algorithm described in Euclid's *Elements* does not use division with remainder, but rather subtracts the smaller number g from the larger one until it becomes smaller than g, and then swaps the two.

Allowing $-\infty$ as a value of a Euclidean function d is a bit annoying and makes our two main examples, integers and univariate polynomials over a field, look different. The proper analogy between $\mathbb{Z}$ and $F[x]$ goes as follows. We can take $d(a) = |a|$ on $\mathbb{Z}$ and $d(a) = 2^{\deg a}$ on $F[x]$, including $d(0) = 0$ in both cases; then $d(ab) = d(a)d(b)$. Or, equivalently, we can take $d(a) = \lfloor \log_2 |a| \rfloor$ on $\mathbb{Z}$ (Exercise 3.5) and $d(a) = \deg a$ on $F[x]$, with $d(0) = -\infty$ in both cases; then $d(ab)$ is $d(a) + d(b)$ (or $d(a) + d(b) + 1$ in $\mathbb{Z}$).

3.2. The astronomical book Āryabhaṭīya, written by Āryabhaṭa in Sanskrit near the end of the fifth century AD, contains an algorithm for computing from two coprime integers $f, g \in \mathbb{N}$ two integers s, t such that $sf + tg = 1$. This problem is also solved in Bachet (1612).

Exercise 3.25 discusses the binary Euclidean Algorithm of Stein (1967). Knuth (1998), already in the second edition, states a binary EEA due to Michael Penk (Algorithm Y in the Answers to Exercises of §4.5.2). Weilert (2000) adapts the binary Euclidean Algorithm to the Gaussian integers.

Although the polynomial version of the (Extended) Euclidean Algorithm is conceptually somewhat simpler, it is much younger (Stevin 1585; Newton 1707, page 38) than the 2000-year old integer algorithm. One reason for this is that we have a more intuitive understanding of integers than we do of polynomials.

3.3. The fact that the number of division steps is maximal for Fibonacci numbers is Lamé's (1844) theorem. The scholarly work of Bach & Shallit (1996) contains more complete historical information about this and many other topics in this book. The interesting paper by Shallit (1994) points to three earlier analyses of the number of divisions in Euclid's algorithm: Reynaud (1824), Finck (1841), and Binet (1841); the latter allows negative remainders, as in Exercise 3.13. Finck's wording *un problème qui* [...] *a pour objet de déterminer le nombre des opérations de la recherche du p.g.c.d. de deux nombres entiers*[1] is a remarkably modern-sounding demand for the analysis of Euclid's algorithm. He gives the inequality $r_{i-1} > 2r_{i+1}$ that we used. Dupré (1846) gives the bounds of about $(\log f)/\log((1+\sqrt{5})/2)$ for the ordinary and $(\log f)/\log(1+\sqrt{2})$ for Binet's Euclidean Algorithm (Exercise 3.30). Much earlier, Schwenter (1636), 86. Auffgab, calls the Euclidean Algorithm for 770020512197390 and 124591930070091, with 32 divisions, the *arithmetical labyrinth*, due to Simon Jacob von Coburg, and points to the Fibonacci numbers as requiring many divisions in the Euclidean Algorithm. (The two large integers are not Fibonacci numbers, and Schwenter says that their Euclidean Algorithm requires 54 divisions; there is a calculation or copying mistake somewhere.) We have $\gcd(F_n, F_m) = F_{\gcd(n,m)}$; see Exercise 3.31.

The average number of division steps in the Euclidean Algorithm for integers was investigated by Heilbronn (1968) and Dixon (1970), and in the binary algorithm (Exercise 3.25) by Brent (1976); see Knuth (1998), §4.5.2, and Shallit (1994) for surveys. Those results were all based on reasonable but unproven assumptions. The question was finally settled by Vallée (2003); she gives average case analyses of several variations, with about $\frac{\pi^2}{6}n$ many divisions on average for the Euclidean Algorithm on n-bit numbers. For polynomials over

[1] a problem that [...] has as its goal to determine the number of operations in computing the gcd of two integers

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

finite fields, Ma & von zur Gathen (1990)) give worst case and average case analyses of several variants of the Euclidean Algorithm.

The fact that two random integers are coprime with probability $6/\pi^2$ is a theorem of Dirichlet (1849). Dirichlet also proves the fact, surprising at first sight, that for fixed a in a division the remainder $r = a \text{ rem } b$, with $0 \leq r < b$, is more likely to be smaller than $b/2$ than larger: If p_a denotes the probability for the former, where $1 \leq b \leq a$ is chosen uniformly at random, then p_a is asymptotically $2 - \ln 4 \approx 61.37\%$. For Dirichlet's theorem, and also the corresponding statement about the probability of being squarefree (due to Gegenbauer 1884), see Hardy & Wright (1985), §§18.5 and 18.6. A heuristic argument goes as follows. A prime p divides a random integer x with probability $1/p$, and neither x nor y with probability $1 - 1/p^2$. Hence $\gcd(x,y) = 1$ happens with probability $\zeta(2)^{-1} = \prod_{p \text{ prime}}(1 - 1/p^2) = 6/\pi^2$; see Notes 18.4 for a discussion of Riemann's zeta function. The value of $\zeta(2)$ was determined by Euler (1734/35b, 1743); see Apostol (1983) for a simple way of calculating this quantity.

3.4. The Euclidean Algorithm 3.14 with monic remainders (for univariate polynomials) appears in the 1969 edition of Knuth (1998), and in Brown (1971).

The calculation of the Bézout coefficients via the EEA in general is in Euler (1748a), §70. See also Notes 6.3. Gauß (1863b), articles 334 and 335, does this for polynomials in $\mathbb{F}_p[x]$, where p is prime.

Exercises.

3.1 Prove that two odd integers whose difference is 32 are coprime.

3.2 Let R be an integral domain. Show that

$$a \sim b \iff (a \mid b \text{ and } b \mid a) \iff \langle a \rangle = \langle b \rangle,$$

where $\langle a \rangle = Ra = \{ra : r \in R\}$ is the ideal generated by a.

3.3 Prove Lemma 3.4. Hint: For (v) and (vi), show that any divisor of the left hand side also divides the right hand side, and vice versa. What are the corresponding statements for the lcm? Are they also true?

3.4* Show that $\gcd(a,b) = 1$ and $\gcd(a,c) = 1$ imply that $\gcd(a,bc) = 1$.

3.5** We consider the following property of a Euclidean function on an integral domain R:

$$d(ab) \geq d(b) \text{ for all } a,b \in R \setminus \{0\}. \tag{9}$$

Our two familiar examples, the degree on $F[x]$ for a field F and the absolute value on $\mathbb{Z}$, both fulfill this property. This exercise shows that every Euclidean domain has such a Euclidean function.

(i) Show that $\delta : \mathbb{Z} \longrightarrow \mathbb{N}$ with $\delta(3) = 2$ and $\delta(a) = |a|$ if $a \neq 3$ is a Euclidean function on $\mathbb{Z}$ violating (9).

(ii) Suppose that R is a Euclidean domain and $D = \{\delta : \delta \text{ is a Euclidean function on } R\}$. Then D is nonempty, and we may define a function $d : R \longrightarrow \mathbb{N} \cup \{-\infty\}$ by $d(a) = \min\{\delta(a) : \delta \in D\}$. Show that d is a Euclidean function on R (called the **minimal Euclidean function**).

(iii) Let δ be a Euclidean function on R such that $\delta(ab) < \delta(b)$ for some $a,b \in R \setminus \{0\}$. Find another Euclidean function δ^* that is smaller than δ. Conclude that the minimal Euclidean function d satisfies (9).

(iv) Show that for all $a,b \in R \setminus \{0\}$ and a Euclidean function d satisfying (9), we have $d(0) < d(a)$, and $d(ab) = d(b)$ if and only if a is a unit.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

(v) Let d be the minimal Euclidean function as in (ii). Conclude that $d(0) = -\infty$ and the group of units of R is $R^\times = \{a \in R \setminus \{0\} : d(a) = 0\}$.

(vi) Prove that $d(a) = \deg a$ is the minimal Euclidean function on $F[x]$ for a field F, and that $d(a) = \lfloor \log_2 |a| \rfloor$ is the minimal Euclidean function on $\mathbb{Z}$, with $d(0) = -\infty$ in both cases.

3.6* (i) Show that each two nonzero elements a, b of a UFD R have a gcd as well as a lcm. You may assume that a normal form on R is given (this is not a restriction, by Exercise 3.9). Hint: First look at the special case $R = \mathbb{Z}$, and use the factorizations of normal(a) and normal(b) into normalized primes.

(ii) Prove that $\gcd(a,b) \cdot \operatorname{lcm}(a,b) = \operatorname{normal}(a \cdot b)$.

(iii) Conclude that $\operatorname{lcm}(a_1, \ldots, a_n) = \operatorname{normal}(a_1 \cdots a_n)$ for any n nonzero elements $a_1, \ldots, a_n \in R$ that are pairwise coprime (you might need Exercise 3.4).

(iv) Is $\gcd(a_1, \ldots, a_n) \cdot \operatorname{lcm}(a_1, \ldots, a_n) = \operatorname{normal}(a_1 \cdots a_n)$ valid for arbitrary $n \in \mathbb{N}$?

3.7* Let R be a Euclidean domain, with a Euclidean function $d : R \longrightarrow \mathbb{N} \cup \{-\infty\}$ that has the additional properties

- $d(ab) = d(a) + d(b)$,
- $d(a+b) \leq \max\{d(a), d(b)\}$, with equality if $d(a) \neq d(b)$,
- d is surjective,

for all $a, b \in R$. Prove that R is a polynomial ring with d as degree function. Proceed as follows:

(i) Prove that $d(a) = -\infty$ if and only if $a = 0$.

(ii) Show that $F = \{a \in R : d(a) \leq 0\}$ is a subfield of R.

(iii) Let $x \in R$ be such that $d(x) = 1$, and prove that every nonzero $a \in R$ has a unique representation

$$a = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

where $n = d(a)$, $a_0, \ldots, a_n \in F$, and $a_n \neq 0$.

Hint: You may find Exercise 3.5 (iv) useful.

3.8 (i) Find normal forms for $\mathbb{Z}[x]$ and for a field F.

(ii) Now let R be an integral domain with a normal form. Prove that $\operatorname{lu}(ua) = u \cdot \operatorname{lu}(a)$ holds for all units $u \in R$ and all $a \in R$. Show that a is normalized if and only if $\operatorname{normal}(a) = a$.

(iii) Prove that $\operatorname{lu}_{R[x]}(f) = \operatorname{lu}_R(f_n)$ for a polynomial $f = \sum_{0 \leq i \leq n} f_i x^i \in R[x]$ with $f_n \neq 0$ defines a normal form on $R[x]$ which extends the given normal form on R. Describe this normal form when R is a field.

(iv) Let $a = bc \in R$ with normalized a, b. Prove that c is normalized if $b \neq 0$, and conclude that $\operatorname{normal}(a/b) = \operatorname{normal}(a)/\operatorname{normal}(b)$ holds for all $a, b \in R$ such that $a \mid b$ and $b \neq 0$.

(v) Starting with the usual normal form $\operatorname{normal}_{\mathbb{Z}}(a) = |a|$ on $\mathbb{Z}$, we can use part (iii) of this exercise in two ways to obtain a normal form on $R = \mathbb{Z}[x,y]$, namely by regarding R either as $\mathbb{Z}[x][y]$ or as $\mathbb{Z}[y][x]$. Determine both normal forms for the polynomial $x - y \in R$.

3.9* The goal of this exercise is to show that every UFD has a normal form.

(i) For a prime $p \in \mathbb{N}$, define $\operatorname{lu}(p) = 1$ if $p \neq 2$ and $\operatorname{lu}(2) = -1$. Show that "lu" can be uniquely extended to a normal form on $\mathbb{Z}$.

(ii) Now let R be an arbitrary UFD and $P \subseteq R$ a complete set of representatives for all non-associate primes of R, so that every prime of R is associate to a unique $p \in P$. (The existence of such a set is guaranteed by the axiom of choice in general.) Then every $r \in R$ can be *uniquely* written as $r = u \prod_{p \in S} p$ for a finite subset $S \subseteq R$ and a unit $u \in R$. Show that $\operatorname{lu}(r) = u$ defines a normal form on R.

3.10 Are there $s, t \in \mathbb{Z}$ such that $24s + 14t = 1$?

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

3.11 For each of the following pairs of integers, find their greatest common divisor using the Euclidean Algorithm:

(i) $34, 21$; (ii) $136, 51$; (iii) $481, 325$; (iv) $8771, 3206$.

3.12 Show that $\{sf+tg : s,t \in \mathbb{Z}\} = \{k \cdot \gcd(f,g) : k \in \mathbb{Z}\}$ holds for all $f,g \in \mathbb{Z}$. (In other words, the two ideals $\langle f,g\rangle$ and $\langle \gcd(f,g)\rangle$ are identical.)

3.13 The Euclidean Algorithm for integers can be slightly speeded up if it is permitted to carry out divisions with negative remainders, so that $r_{i-1} = r_i q_i + r_{i+1}$ with $-|r_i/2| < r_{i+1} \leq |r_i/2|$. Do the four examples in Exercise 3.11 using this method.

3.14 Use the Extended Euclidean Algorithm to find $\gcd(f,g)$, for $f,g \in \mathbb{Z}_p[x]$ in each of the following examples (arithmetic in $\mathbb{Z}_p = \{0,\ldots,p-1\}$ is done modulo p). In each case compute the corresponding polynomials s and t such that $\gcd(f,g) = sf+tg$.

(i) $f = x^3+x+1, g = x^2+x+1$ for $p=2$ and $p=3$.

(ii) $f = x^4+x^3+x+1$, $g = x^3+x^2+x+1$ for $p=2$ and $p=3$.

(iii) $f = x^5+x^4+x^3+x+1$, $g = x^4+x^3+x^2+x+1$ for $p=5$.

(iv) $f = x^5+x^4+x^3-x^2-x+1$, $g = x^3+x^2+x+1$ for $p=3$ and $p=5$.

3.15 Show that the s_i and t_i in the traditional Extended Euclidean Algorithm for inputs $f,g \in \mathbb{Z}$ with $f > g > 0$ alternate in sign, so that s_{2i} and t_{2i-1} are positive and s_{2i+1} and t_{2i} are negative for all admissible values of $i \geq 1$. Conclude that $0 = s_1 < 1 = s_2 \leq |s_3| < |s_4| < \cdots < |s_{\ell+1}|$ and $0 = t_0 < 1 = t_1 \leq |t_2| < |t_3| < \cdots < |t_{\ell+1}|$.

3.16 Let R be a Euclidean domain, $a,b,c \in R$, and $\gcd(a,b) = 1$. Prove the following:

(i) $a \mid bc \Longrightarrow a \mid c$,

(ii) $a \mid c$ and $b \mid c \Longrightarrow ab \mid c$.

Hint: You may want to use the fact that the Extended Euclidean Algorithm computes $s,t \in R$ such that $sa+tb = 1$.

3.17 Prove that $\mathbb{Z}[x]$ is not a Euclidean domain. Hint: If it were, then we could compute $s,t \in \mathbb{Z}[x]$ such that $s \cdot 2 + t \cdot x = \gcd(2,x)$, using the Extended Euclidean Algorithm.

3.18* Let $R = F[x]$ for a field F and

$$S = \bigcup_{\ell \geq 1} \Big((F \setminus \{0\})^{\ell+1} \times (R \setminus \{0\})^2 \times \{q \in R : \deg q > 0,\ q \text{ monic}\}^{\ell-1} \Big).$$

The **Euclidean representation** of a pair $(f,g) \in (R \setminus \{0\})^2$ with $\deg f \geq \deg g$ is defined as the list $(\rho_0,\ldots,\rho_\ell, r_\ell, q_1,\ldots,q_\ell) \in S$ formed from the results of the Euclidean Algorithm. Show that the map

$$\{(f,g) \in R^2 : \deg f \geq \deg g \text{ and } g \neq 0\} \longrightarrow S$$

which maps a pair of polynomials (f,g) to its Euclidean representation is a bijection.

3.19* (i) Show that the **norm** $N : \mathbb{Z}[i] \longrightarrow \mathbb{N}$ with $N(\alpha) = \alpha\overline{\alpha}$ on the ring of Gaussian integers $\mathbb{Z}[i]$ is a Euclidean function. Hint: Consider the exact quotient of two Gaussian integers $\alpha, \beta \in \mathbb{Z}[i]$ in $\mathbb{C}$.

(ii) Show that the units in $\mathbb{Z}[i]$ are precisely the elements of norm 1 and enumerate them.

(iii) Prove that there is no multiplicative normal form on $\mathbb{Z}[i]$ which extends the usual normal form $\text{normal}(a) = |a|$ on $\mathbb{Z}$. Hint: Consider $\text{normal}((1+i)^2)$. Why is $\text{normal}(a+ib) = |a| + i|b|$ for $a,b \in \mathbb{Z}$ *not* a normal form?

(iv) Compute all greatest common divisors of 6 and $3+i$ in $\mathbb{Z}[i]$ and their representations as a linear combination of 6 and $3+i$.

(v) Compute a gcd of 12277 and $399+20i$.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

3.20* Let $x_1, x_2, \ldots$ be countably many indeterminates over $\mathbb{Z}$, $R = \mathbb{Z}[x_1, x_2, \ldots]$,

$$Q_i = \begin{pmatrix} 0 & 1 \\ 1 & x_i \end{pmatrix} \in R^{2\times 2}$$

for $i \geq 1$, and $R_i = Q_i \cdots Q_1$. We define the ith **continuant polynomial** $c_i \in R$ recursively by $c_0 = 0$, $c_1 = 1$, and $c_{i+1} = c_{i-1} + x_i c_i$ for $i \geq 1$. Then $c_i \in \mathbb{Z}[x_1, \ldots, x_{i-1}]$ for $i \geq 1$.

(i) List the first 10 continuant polynomials.

(ii) Let T be the "shift homomorphism" $T x_i = x_{i+1}$ for $i \geq 1$. Show that $c_{i+2}(0, x_1, x_2, \ldots, x_i) = T c_i$ for $i \geq 0$.

(iii) Show that $R_i = \begin{pmatrix} T c_{i-1} & c_i \\ T c_i & c_{i+1} \end{pmatrix}$ for $i \geq 1$.

(iv) Show that $\det R_i = (-1)^i$, and conclude that $\gcd(c_i, c_{i+1}) = 1$ for $i \geq 0$.

(v) Let D be a Euclidean domain and $r_i, q_i, s_i, t_i \in D$ for $0 \leq i \leq \ell$ the results of the *traditional* Extended Euclidean Algorithm for r_0, r_1. Show that

$$\begin{aligned} s_i &= c_{i-1}(-q_2, \ldots, -q_{i-1}) = (-1)^i c_{i-1}(q_2, \ldots, q_{i-1}), \\ t_i &= c_i(-q_1, \ldots, -q_{i-1}) = (-1)^{i-1} c_i(q_1, \ldots, q_{i-1}) \end{aligned}$$

for $1 \leq i \leq \ell$.

(vi) Write a MAPLE program that implements the traditional Extended Euclidean Algorithm and additionally computes all continuants $c_i(q_{\ell-i+2}, \ldots, q_\ell)$ for $r_0 = x^{20}$ and $r_1 = x^{19} + 2x^{18} + x$ in $\mathbb{Q}[x]$, where $q_1, \ldots, q_\ell$ are the quotients in the traditional Extended Euclidean Algorithm.

3.21 (i) Prove Lemma 3.10 (6) for the traditional EEA 3.6. Hint: Since q_1 may be constant, it is wise to start the induction with $i = 3$ and show the cases $i = 1$ and $i = 2$ separately.

(ii) Prove Lemma 3.10 for the Extended Euclidean Algorithm 3.14.

3.22 (i) Show that for polynomials $f, g \in F[x]$ of degrees $n \geq m$, where F is a field, computing all entries s_i in the traditional Extended Euclidean Algorithm from the quotients q_i takes at most $2m^2 + 2m$ additions and multiplications in F. Hint: Exhibit the bound for the normal case and prove that this is the worst case.

(ii) Prove that the corresponding estimate for the Extended Euclidean Algorithm is $2m^2 + 3m + 1$.

3.23 Prove Lemma 3.12. Hint: Use Lemma 3.8 and Exercise 3.15.

3.24* Prove Theorem 3.13.

3.25* We consider the following recursive algorithm for computing the gcd of two integers.

ALGORITHM 3.17 Binary Euclidean Algorithm.
Input: $a, b \in \mathbb{N}_{>0}$.
Output: $\gcd(a, b) \in \mathbb{N}$.

1. **if** $a = b$ **then return** a
2. **if** both a and b are even **then return** $2 \cdot \gcd(a/2, b/2)$
3. **if** exactly one of the two numbers, say a, is even **then return** $\gcd(a/2, b)$
4. **if** both a and b are odd and, say, $a > b$, **then return** $\gcd((a-b)/2, b)$

(i) Run the algorithm on the examples of Exercise 3.11.

(ii) Prove that the algorithm works correctly.

(iii) Find a "good" upper bound on the recursion depth of the algorithm, and show that it takes $O(n^2)$ word operations on inputs of length at most n.

(iv) Modify the algorithm so that it additionally computes $s, t \in \mathbb{N}$ such that $sa + tb = \gcd(a, b)$.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

3.26* Adapt the algorithm from Exercise 3.25 to polynomials over a field. Hint: Start with $\mathbb{F}_2[x]$.

3.27 Let F_n and F_{n+1} be consecutive terms in the Fibonacci sequence. Show that $\gcd(F_{n+1}, F_n) = 1$.

3.28 (i) Prove the formula

$$F_n = \frac{1}{\sqrt{5}}(\phi_+^n - \phi_-^n) \text{ for } n \in \mathbb{N} \tag{10}$$

for the Fibonacci numbers, where $\phi_+ = (1+\sqrt{5})/2 \approx 1.618$ is the golden ratio and $\phi_- = -1/\phi_+ = (1-\sqrt{5})/2 \approx -0.618$. Conclude that F_n is the nearest integer to $\phi_+^n/\sqrt{5}$ for all n.

(ii) For $n \in \mathbb{N}_{>0}$, let $k_n = [1, \ldots, 1]$ be the continued fraction of length n with all entries equal to 1 (Section 4.6). Prove that $k_n = F_{n+1}/F_n$, and conclude that $\lim_{n\longrightarrow\infty} k_n = \phi_+$.

3.29* This continues Exercise 3.28.

(i) Let $h = \sum_{n\geq 0} F_n x^n \in \mathbb{Q}[[x]]$ be the formal power series whose coefficients are the Fibonacci numbers. Derive a linear equation for h from the recursion formula for the Fibonacci numbers and solve it for h. (It will turn out that h is a rational function in x.)

(ii) Compute the partial fraction expansion (Section 5.11) of h and use it to prove (10) again by employing the formula $\sum_{n\geq 0} x^n = 1/(1-x)$ for the geometric series and comparing coefficients.

3.30* In the least absolute remainder variant of the Euclidean Algorithm for integers (Exercise 3.13), all quotients q_i (with the possible exception of q_1) are at least two in absolute value. Thus the nonnegative integers with the largest possible number of division steps in this variant, that is, the analog of the Fibonacci numbers in Lamé's theorem, are recursively defined by

$$G_0 = 0, \quad G_1 = 1, \quad G_{n+1} = 2G_n + G_{n-1} \text{ for } n \geq 1.$$

(i) Find a closed form expression similar to (10) for G_n. Hint: Proceed as in Exercise 3.29.

(ii) Derive a tight upper bound on the length ℓ of the least absolute remainder Euclidean Algorithm for two integers $f, g \in \mathbb{N}$ with $f > g$ in terms of $\log g$, and compare it to (8).

3.31* For $n \in \mathbb{N}$, let F_n be the nth Fibonacci number, with $F_0 = 0$ and $F_1 = 1$. Prove or disprove that the following properties hold for all $n, k \in \mathbb{N}$.

(i) $F_{n+k+1} = F_n F_k + F_{n+1} F_{k+1}$,

(ii) F_k divides F_{nk},

(iii) $\gcd(F_{nk+1}, F_k) = 1$ if $k \geq 1$ (hint: Exercise 3.27),

(iv) $F_n \text{ rem } F_k = F_{n \text{ rem } k}$ if $k \geq 1$,

(v) $\gcd(F_n, F_k) = \gcd(F_k, F_{n \text{ rem } k})$ if $k \geq 1$ (hint: Exercise 3.16),

(vi) $\gcd(F_n, F_k) = F_{\gcd(n,k)}$.

(vii) Conclude from (i) that F_n can be calculated with $O(\log n)$ arithmetic operations in $\mathbb{Z}$.

(viii) Generalize your answers to **Lucas sequences** $(L_n)_{n\geq 0}$ of the form $L_0 = 0$, $L_1 = 1$, and $L_{n+2} = aL_{n+1} + L_n$ for $n \in \mathbb{N}$, where $a \in \mathbb{Z}$ is a fixed constant.

3.32* We define the sequence $f_0, f_1, f_2, \ldots \in \mathbb{Q}[x]$ of monic polynomials by

- $\gcd(f_n, f_{n-1}) = 1$ for $n \geq 1$,
- for every $n \geq 1$ the number of division steps in the Euclidean Algorithm for (f_n, f_{n-1}) is n, and all quotients are equal to x.

(i) What are the remainders in the Euclidean Algorithm for f_n and f_{n-1}? What are the ρ_i? Find a recursion for the f_n. What is the degree of f_n?

(ii) What is the connection between the f_n and the Fibonacci numbers?

(iii) State and prove a theorem saying that the number of division steps in the Euclidean Algorithm for the pair (f_n, f_{n-1}) is maximal. Make explicit what you mean by maximal.

3.33 Let R be a ring, and $f, g, q, r \in R[x]$ with $g \neq 0$, $f = qg + r$, and $\deg r < \deg g$. Prove that q and r are unique if and only if $\text{lc}(g)$ is not a zero divisor.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press

Die Musik hat viel Aehnlichkeit mit der Algeber.[1]
Novalis (1799)

Ein Mathematiker, der nicht etwas Poet ist,
wird nimmer ein vollkommener Mathematiker sein.[2]
Karl Theodor Wilhelm Weierstraß (1883)

There remain, therefore, algebra and arithmetic
as the only sciences, in which we can carry on
a chain of reasoning to any degree of intricacy,
and yet preserve a perfect exactness and certainty.
David Hume (1739)

The science of algebra, independently of any of its uses, has all the advantages which belong to mathematics in general as an object of study, and which it is not necessary to enumerate. Viewed either as a science of quantity, or as a language of symbols, it may be made of the greatest service to those who are sufficiently acquainted with arithmetic, and who have sufficient power of comprehension to enter fairly upon its difficulties.
Augustus De Morgan (1837)

وهو تقريب لا تحقيق
ولا يقف احد علي حقيقة ذلك ولا يعلم دورها الا الله
لان الخط ليس بمستقيم فيوقف علي حقيقته
وانما قيل ذلك تقريب كما قيل في جذر الاصم
انه تقريب لا تحقيق لان جذره لا يعلمه الا الله[3]
Abū Jaʿfar Muḥammad bin Mūsā al-Khwārizmī (c. 830)

[1] Music has much resemblance to algebra.

[2] A mathematician who is not somewhat of a poet will never be a perfect mathematician.

[3] And this is an approximation and not a precise determination [of π]. Nobody can determine the exact value of that and know the circumference, except God. For this curve [the circle] is not straight and cannot be determined except approximately. That is called an approximation, just as the root of a number is an approximation and not the exact value; nobody knows it except God.

https://doi.org/10.1017/CBO9781139856065.005 Published online by Cambridge University Press